

TP14 – ÊTRE À LA MODE

1. CODE DÉJÀ TOUT PRÊT

Une archive contenant du code pré-écrit en OCaml et les fichiers de données est disponible dans tous les points de vente habituels. Vous en avez besoin pour traiter ce sujet.

2. BASE DU MNIST

La base de données du MNIST est un jeu de données contenant des chiffres manuscrits, que tout le monde utilise en apprentissage comme jeu de données usuel. Cette base fournit en fait deux jeux de données :

- un jeu de 60000 images, avec les chiffres associés, destinées à l'entraînement,
- un jeu de 10000 images, avec les chiffres associés, destinées aux tests.

Les images sont des carrés de 28 pixels de côté.

Ces données sont accessibles à l'adresse <http://yann.lecun.com/exdb/mnist/>, où l'on trouvera également une description précise du format des fichiers. Le format est très simple à lire : je vous invite très fortement à implémenter vous-mêmes à titre d'exercice des fonctions de lecture des fichiers IDX en C et en OCaml (mais ce n'est pas l'objet du TP aujourd'hui).

La base de données Fashion MNIST (<https://github.com/zalando-research/fashion-mnist>) suit exactement le même format que la base d'origine du MNIST sauf que les images représentent des vêtements au lieu de représenter des chiffres. C'est avec cette base de données que l'on travaille dans ce sujet.

Vous avez à votre disposition un module `Mnist`. Lisez son interface dans `mnist.mli`. Ses fonctions permettent de lire des données contenues dans un fichier IDX. La lecture d'une image donne des données dans un tableau d'entiers à une dimension, contenant l'image ligne par ligne.

Question 1. Écrire une fonction qui affiche une représentation d'une image à l'écran, à partir de son index dans la base du MNIST. Dans les données, chaque pixel est un entier compris entre 0 (blanc) et 255 (noir). On peut choisir un jeu de caractères qui représentera chaque luminosité : le fichier `grayscale_70_levels.txt` vous suggère une liste de 70 caractères classiques pour représenter les tons allant du blanc au noir en ASCII art. Vous pouvez utiliser n'importe quelle autre représentation.

On va tenter de lire les données de manière paresseuse, c'est-à-dire sans nécessairement toujours charger les 60000 symboles en mémoire. Au lieu d'utiliser des listes, on utilise le type OCaml `Seq.t` du module `Seq`. Il est défini de la façon suivante :

```
type 'a t = unit -> 'a node
type 'a node =
  | Nil
  | Cons of 'a * 'a t
```

OCAML

Un `Seq.t` est une fonction, qui prend en paramètre un `unit`. L'élément suivant de la séquence n'est donc obtenu qu'en le demandant explicitement par un appel de fonction. Cette appel de fonction renvoie soit `Nil` pour indiquer que la séquence est terminée, soit `Cons(x, f)` où `x` est l'élément suivant et `f` est une fonction qui, quand on l'appellera, donnera la suite de la séquence.

3. FILE DE PRIORITÉ

Question 2. Lisez l'interface du module `PrioQueue` qui se trouve dans le fichier fourni `prioQueue.mli`. Ce module vous donne une implémentation des files de priorité non mutables avec des arbres en tas minimum.

La racine est l'élément minimal pour la fonction de comparaison fournie à la création. Cette fonction respecte la convention usuelle de la fonction `Stdlib.compare` de la bibliothèque standard d'OCaml. En vous permettant de choisir votre fonction de

comparaison, le module vous permet d'utiliser la notion d'ordre qui vous convient pour vos données.

4. k PLUS PROCHES VOISINS NAÏF

Le but de cette partie est d'écrire un module `Knn` dans le fichier `knn.ml`, dont l'interface est fixée par `knn.mli`. Vous pouvez vous lancer directement dans l'écriture de ce module sans autre indication, ou bien vous laisser guider par les questions suivantes, qui décomposent le travail.

Question 3. Écrire une fonction `count_item` qui compte le nombre d'occurrences d'un élément dans une liste.

Question 4. Écrire une fonction `most_frequent` qui détermine, dans une liste non vide, un élément dont le nombre d'occurrences dans la liste est maximal.

Question 5. Écrire une fonction `euclidean_dist : float array → float array → float` qui calcule la distance euclidienne entre deux vecteurs de $\mathbb{R}^d$. Les tableaux donnés en entrée sont supposés être de même longueur.

Question 6. Écrire une fonction `mnist_list` qui prend en paramètre deux valeurs de type `Mnist.idx`, l'une pour le fichier d'images et l'autre pour le fichier de réponses, et qui construit une liste de couples formées d'une image et de la réponse correspondante (si vous avez compris les séquences, fabriquez une séquence plutôt qu'une liste).

Question 7. Écrire une fonction `classify` qui prend en paramètre la liste précédente, un entier `k` et une image du MNIST, qui détermine les `k` plus proches voisins de cette image et renvoie une classe majoritaire parmi ces voisins. On implémentera l'algorithme naïf qui parcourt la liste de toutes les données.

Question 8. Tester votre fonction sur les images du jeu de tests : pensez à vous créer un fichier, par exemple `main.ml` qui contient une fonction principale qui lancera tous vos tests. Ce jeu de tests étant fourni avec les réponses, déterminer en fonction de la valeur de `k` le taux d'erreur de votre reconnaissance. Quelle valeur de `k` est la plus adaptée sur ces tests ?

5. ARBRES DIMENSIONNELS

Question 9. Proposer un type OCaml pour représenter un arbre dimensionnel. (Indication : il n'y a à priori rien de très malin à penser, la réponse est immédiate. Il faut juste penser à stocker les bonnes informations à chaque nœud.)

Question 10. Écrire une fonction `dimensional_tree` qui crée l'arbre dimensionnel à partir du jeu de données de Fashion MNIST.

Question 11. Améliorer la fonction `init` afin qu'elle construise l'arbre `k`-dimensionnel puis améliorer la fonction `classify` pour qu'elle utilise cet arbre. Discuter le gain de performance.